APCS 程式設計系列 · U09 · 實作中高級 · Python版

堆疊、佇列與內建資料結構 重點講義

stack 後進先出 · 括號配對與後序運算式 · queue 先進先出 · deque · 優先佇列 · map / set 計數去重 · 陣列模擬

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）

1. 堆疊 stack：後進先出（LIFO）

堆疊像一疊盤子：**最後放上去的，最先被拿走**。Python 直接拿 **list** 當堆疊：append 放到最上面（尾巴）、st[-1] 看最上面、pop() 拿掉並回傳最上面。

Python stack 的三個動作

```python
st = []
for x in [1, 2, 3]:
    st.append(x)
print(st[-1], end=" ")    # 3：最後放進去的在最上面
st.pop()                  # 拿掉 3（Python 的 pop 會回傳 3）
print(st[-1], len(st))
```

輸入	輸出
（不需要輸入）	3 2 2

Python 括號配對：遇到右括號就看堆疊頂端

```python
s = input().strip()
pair = {")": "(", "]": "[", "}": "{"}
st = []
ok = True
for c in s:
    if c in "([{":
        st.append(c)
    elif not st or st[-1] != pair[c]:    # 空的、或配錯
        ok = False
        break
    else:
        st.pop()
if st:                # 還有左括號沒配到
    ok = False
print("Yes" if ok else "No")
```

輸入	輸出
([]{()})	Yes

陷阱提醒 括號配對最常漏的兩件事

【錯】堆疊已經空了還去看 st[-1]：會發生 IndexError。一定先檢查 not st。

【錯】全部讀完就說 Yes：像「((」這種，堆疊裡還剩左括號，一樣不合法。

【對】堆疊的應用：括號配對、後序運算式、復原（上一步）、深度優先搜尋。

2. 後序運算式：用堆疊算，不需要括號

後序運算式把運算符號寫在兩個數字**後面**：3 + 4 寫成 3 4 +。計算規則只有兩條：**讀到數字就放進堆疊；讀到運算符號，就拿出兩個數算完，結果放回去。**

讀到	動作	堆疊內容（左邊是底）
5 1 2	依序放進去	5 1 2
+	拿出 2、1，算 1 + 2 = 3	5 3
4	放進去	5 3 4
*	拿出 4、3，算 3 * 4 = 12	5 12
+	拿出 12、5，算 5 + 12 = 17	17
3 -	放進 3；拿出 3、17，算 17 - 3 = 14	14（答案）

Python 後序運算式求值

```python
st = []
for t in input().split():
    if t in ("+", "-", "*"):
        b = st.pop()        # 先出來的是右邊！
        a = st.pop()
        if t == "+":
            st.append(a + b)
        elif t == "-":
            st.append(a - b)
        else:
            st.append(a * b)
    else:
        st.append(int(t))   # 數字直接放進去
print(st[-1])
```

輸入	輸出
5 1 2 + 4 * + 3 -	14

陷阱提醒

【錯】拿出來的順序寫反：先拿出來的 b 是**右邊**，要算 a - b。寫成 b - a，減法、除法全錯。

【對】中序轉後序的檢查法：把後序從左到右用上面的規則算一次，看結果跟原式一不一樣。

- 中序 (3 + 4) * 2 的後序是 3 4 + 2 *；前序（符號在前）是 * + 3 4 2。

3. 佇列 queue：先進先出（FIFO）

佇列就是排隊：**先來的先處理**。Python 用 **collections.deque**：append 排到尾巴、q[0] 看最前面、popleft() 讓最前面離開。

Python 丟牌遊戲：丟一張、下一張移到最底

```python
from collections import deque
n = int(input())
q = deque(range(1, n + 1))
while len(q) > 1:
    print(q.popleft(), end=" ")   # 最上面那張丟掉
    q.append(q.popleft())          # 下一張移到最底下
print()
print(q[0])
```

輸入	輸出
7	1 3 5 7 4 2 6

陷阱提醒 佇列為什麼不能用 list.pop(0)？

【錯】list.pop(0) 拿掉第一個：後面每個元素都要往前搬一格，一次 O(n)；insert(0, x) 也一樣。

- 做 n 次就是 $O(n^2)$：n = 2×10^5 時大約要搬 2×10^{10} 次，一定超時。

【對】deque 的 popleft()、appendleft() 都是 O(1)。

4. deque 與優先佇列

想做的事	C++	Python
堆疊：放 / 看頂端 / 拿	push(x) top() pop()	append(x) st[-1] pop()
佇列：排入 / 看前面 / 拿	push(x) front() pop()	append(x) q[0] popleft()
deque：前面放 / 後面拿	push_front(x) pop_back()	appendleft(x) pop()
優先佇列：放 / 看最優先 / 拿	push(x) top() pop()	heappush(h, x) h[0] heappop(h)
優先佇列預設誰先出？	**大的**先出	**小的**先出

優先佇列不管放進去的順序，每次拿出來的都是「最優先」的那一個，放入、拿出都是 O(log n)。適合「每次都要取目前最大（最小）值」的題目，例如叫號、排程、合併。

Python priority_queue 與 heapq：誰先出來？

```python
import heapq
big, small = [], []
for x in [5, 1, 8, 3]:
    heapq.heappush(small, x)   # heapq 一律小的先出
    heapq.heappush(big, -x)    # 存負數 → 大的先出
print(-big[0], small[0])
```

輸入	輸出
（不需要輸入）	8 1

陷阱提醒

【錯】以為 heapq 可以設定成大的先出：heapq 一律**小的**先出，要大的先出就存負數，拿出來再變回正的。

【錯】同分要「先到先出」時，只放分數是不夠的，要把「到達順序」一起放進去比較。

【錯】heapq 的 h 本身不是排好序的串列，只保證 h[0] 最小；要依序取出，一定要反覆 heappop。

5. map / set / dict / Counter：計數與去重

用途	C++	Python
計數（key → 次數）	map<string, int>：依 key 由小到大	dict 或 Counter：依放入順序
沒出現過的 key	cnt[k]++ 自動從 0 開始	Counter 查到 0；dict 要用 get(k, 0)
去重	set：自動排好序	set：**沒有固定順序**
雜湊版（更快、無序）	unordered_map / unordered_set	（dict、set 本身就是雜湊）

Python 計數：每個單字出現幾次（依字典序印）

```python
import sys
from collections import Counter
cnt = Counter(sys.stdin.read().split())
for k in sorted(cnt):        # 排序之後，順序才固定
    print(k, cnt[k])
```

輸入	輸出
banana apple cherry apple banana apple	apple 3 banana 2 cherry 1

陷阱提醒

【錯】直接印出一個 set：順序不固定。題目要求順序時，一律 sorted() 之後再印。

【錯】dict 直接寫 d[k] += 1：k 沒出現過會 KeyError。用 d.get(k, 0) + 1 或 Counter。

【對】排序條件有兩層（例如票數多的在前、同票依名字）：C++ 把票數取負號放進 pair 排序，Python 用 key=lambda t: (-t[1], t[0])。

6. Python 題本：用 list 當堆疊與佇列

Python 題本常直接用 list：**pop() 從尾巴拿 = 堆疊；pop(0) 從頭拿 = 佇列**。識讀題只要看清楚是從哪一頭拿。

Python 題本：list 當堆疊和佇列

```python
st = [4, 7, 1]
print(st.pop(), end=" ")   # 從尾巴拿 → 1（堆疊）
q = [4, 7, 1]
print(q.pop(0), end=" ")   # 從頭拿 → 4（佇列，但 O(n)）
print(len(st), len(q))
```

輸入	輸出
（不需要輸入）	1 4 2 2

追蹤陣列模擬的四個重點

- **pop() 與 pop(0)**：不加參數從尾巴拿，加 0 從頭拿；拿走之後 list 會變短。
- **索引 -1**：st[-1] 是最後一個（堆疊頂端），q[0] 是第一個（佇列前端）。
- **deque** 前後都能放、都能拿：append、appendleft、pop、popleft。
- 追蹤時把每一步的 list 內容寫在旁邊，不要只在腦中想。

讀完這份，你應該要會

- □ 會用堆疊做括號配對，記得檢查「堆疊空了」和「最後還有剩」
- □ 會用堆疊算後序運算式，知道先拿出來的是右邊的數
- □ 佇列一律用 collections.deque，不用 list.pop(0)
- □ 知道 heapq 小的先出，要大的先出就存負數
- □ 計數用 map / dict / Counter；輸出順序有規定時，一定先排序
- □ 看得懂 Python 題本用 list 的 pop() 和 pop(0) 模擬的堆疊與佇列

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 jenfoo2021

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）